

# Security Documentation

---

## Overview

---

This document provides detailed information about the security features and considerations of the Vault Secure Session Management system.

## Security Architecture

---

### 1. Authentication Layer

#### Password Hashing

- **Algorithm:** Argon2id (winner of the Password Hashing Competition)
- **Parameters:** Configurable memory cost, time cost, and parallelism
- **Salt:** Cryptographically secure random salt for each password
- **Output:** 32-byte hash with embedded salt and parameters

#### Account Protection

- **Failed Attempt Tracking:** Configurable threshold for account lockout
- **Lockout Duration:** Configurable lockout period after failed attempts
- **Account Status:** Active/inactive status with manual override capability

### 2. Session Management Layer

#### Session Creation

- **Token Generation:** 64-character cryptographically secure random tokens
- **Session Data:** Encrypted using AES-256-GCM with unique nonce per session
- **Metadata Tracking:** IP address, user agent, creation time, last access

#### Session Validation

- **Expiration Checking:** Automatic validation of session expiry
- **Renewal Mechanism:** Secure session renewal without re-authentication
- **Concurrent Limits:** Configurable maximum concurrent sessions per user

#### Session Storage

- **Encryption:** All session data encrypted at rest
- **Key Management:** Secure key generation and storage
- **Memory Protection:** Sensitive data protected using secrecy crate

### 3. Cryptographic Layer

#### Key Derivation

- **Argon2id Configuration:**
  - Memory Cost: 64 MiB (configurable)
  - Time Cost: 3 iterations (configurable)
  - Parallelism: 4 threads (configurable)
  - Output Length: 32 bytes

## Encryption

- **Algorithm:** AES-256-GCM (Authenticated Encryption)
- **Key Size:** 256-bit keys
- **Nonce:** 96-bit random nonce per encryption
- **Authentication:** Built-in authentication tag

## Random Number Generation

- **Source:** Operating system's cryptographically secure RNG
- **Usage:** Token generation, salt generation, nonce generation

## 4. Memory Protection

### Secure Memory Handling

- **Secrecy Crate:** Wrapper types for sensitive data
- **Automatic Zeroization:** Memory cleared on drop
- **Debug Protection:** Sensitive data not exposed in debug output

### Memory Wiping

- **Multi-pass Wiping:** Zeros, ones, random, then zeros
- **Compiler Barriers:** Prevents optimization of memory clearing
- **Secure Deallocation:** Ensures sensitive data is not recoverable

## Security Configuration

### Default Security Settings

```
SecurityConfig {
    max_failed_attempts: 5,           // Account lockout after 5 failed attempts
    lockout_duration_secs: 900,       // 15-minute lockout period
    session_timeout_secs: 3600,       // 1-hour session timeout
    max_concurrent_sessions: 3,       // Maximum 3 concurrent sessions
    enable_memory_protection: true,    // Enable secure memory protection
    session_renewal_interval_secs: 1800, // 30-minute renewal interval
}
```

### Argon2 Configuration

```
Argon2Config {
    memory_cost: 65536, // 64 MiB memory usage
    time_cost: 3,       // 3 iterations
    parallelism: 4,      // 4 parallel threads
    output_length: 32,   // 32-byte output
}
```

## Threat Model

### Threats Addressed

#### 1. Password Attacks

- Brute force attacks (mitigated by Argon2 and account lockout)
- Dictionary attacks (mitigated by strong hashing)
- Rainbow table attacks (mitigated by unique salts)

## 2. Session Attacks

- Session hijacking (mitigated by secure tokens and HTTPS requirement)
- Session fixation (mitigated by token regeneration)
- Session replay (mitigated by expiration and renewal)

## 3. Memory Attacks

- Memory dumps (mitigated by secure memory protection)
- Swap file exposure (mitigated by memory locking where available)
- Debug information leakage (mitigated by secrecy wrappers)

## 4. Timing Attacks

- Password verification timing (mitigated by constant-time operations)
- Session lookup timing (mitigated by consistent validation flow)

## Threats Not Addressed

### 1. Network-level Attacks

- Man-in-the-middle attacks (requires HTTPS at application level)
- Network sniffing (requires transport layer security)

### 2. Application-level Attacks

- CSRF attacks (requires application-level CSRF protection)
- XSS attacks (requires application-level input validation)

### 3. Infrastructure Attacks

- Server compromise (requires infrastructure security)
- Database attacks (requires database security)

## Security Best Practices

---

### Deployment Security

#### 1. Transport Security

- Always use HTTPS in production
- Implement HTTP Strict Transport Security (HSTS)
- Use secure cookie flags for session tokens

#### 2. Key Management

- Store encryption keys securely (consider HSM)
- Rotate encryption keys regularly
- Use separate keys for different environments

#### 3. Monitoring and Logging

- Log authentication events
- Monitor failed login attempts
- Alert on suspicious patterns
- Regular security audits

### Configuration Tuning

#### 1. Argon2 Parameters

- Adjust based on available hardware
- Target 100-500ms for password hashing
- Balance security vs. performance

## 2. Session Settings

- Set appropriate timeout based on risk
- Limit concurrent sessions based on use case
- Configure renewal interval for long-running sessions

## 3. Account Lockout

- Balance security vs. usability
- Consider progressive delays
- Implement unlock mechanisms

# Compliance Considerations

---

## Standards Compliance

- **OWASP**: Follows OWASP authentication guidelines
- **NIST**: Aligns with NIST password guidelines (SP 800-63B)
- **GDPR**: Supports data protection through secure handling

## Audit Trail

- Authentication events logged
- Session lifecycle tracked
- Failed attempts recorded
- Administrative actions logged

# Security Testing

---

## Recommended Tests

### 1. Authentication Testing

- Password strength validation
- Account lockout functionality
- Failed attempt tracking

### 2. Session Testing

- Session expiration
- Concurrent session limits
- Session renewal

### 3. Cryptographic Testing

- Key derivation verification
- Encryption/decryption testing
- Random number quality

## Penetration Testing

Regular penetration testing should include:

- Authentication bypass attempts
- Session manipulation attacks
- Timing attack analysis
- Memory analysis (where possible)

# Incident Response

---

## Security Incident Handling

### 1. Detection

- Monitor for unusual patterns
- Alert on multiple failed attempts
- Track session anomalies

### 2. Response

- Immediate account lockout if needed
- Session termination procedures
- Investigation protocols

### 3. Recovery

- Account unlock procedures
- Password reset mechanisms
- Session cleanup

# Updates and Maintenance

---

## Security Updates

- Regular dependency updates
- Security patch management
- Vulnerability assessment
- Code review processes

## Key Rotation

- Encryption key rotation schedule
- Session key management
- Password hash migration (if needed)

## Contact

---

For security-related questions or to report vulnerabilities, please contact the security team.